

1 Experiment 2: Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

1.1 Objective

To implement a Multi-Layer Perceptron (MLP) using TensorFlow/Keras for multi-class image classification on the Fashion-MNIST dataset. The experiment includes image preprocessing, model construction, training, evaluation, and automated hyperparameter optimization.

1.2 Learning Outcomes

After completing this experiment, the following learning outcomes were achieved:

- Understand the architecture and working principle of an MLP.
- Preprocess image data for input to dense neural network layers.
- Construct and train an MLP using TensorFlow/Keras.
- Evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
- Perform automated hyperparameter optimization using RandomizedSearchCV.
- Identify and recommend the best-performing MLP configuration.

1.3 Theory

A Multi-Layer Perceptron (MLP) is a feed-forward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. Each neuron performs a weighted summation followed by a nonlinear activation function.

The mathematical operation at layer l is given by:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)} \quad (1)$$

and

$$a^{(l)} = f(z^{(l)}) \quad (2)$$

For multi-class classification, the output layer uses the Softmax activation function:

$$\hat{y} = \text{Softmax}(z) \quad (3)$$

The categorical cross-entropy loss is:

$$L = - \sum_i y_i \log(\hat{y}_i) \quad (4)$$

The recommended baseline architecture for this experiment is:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

1.4 Dataset

The Fashion-MNIST dataset was used for the experiment. It consists of grayscale images of clothing items belonging to ten different classes.

Table 1: Fashion-MNIST Dataset Characteristics

Property	Value
Training Images	60,000
Testing Images	10,000
Number of Classes	10
Image Dimensions	28×28
Input Features after Flattening	784

1.5 Experimental Procedure

The following steps were performed:

1. Load the Fashion-MNIST dataset.
2. Explore the dataset dimensions and class distribution.
3. Display sample images.
4. Flatten each 28×28 image into a 784-dimensional vector.
5. Normalize pixel values to the range $[0, 1]$.
6. Convert class labels into one-hot encoded vectors.
7. Construct the baseline MLP.
8. Compile the model using Adam optimizer and categorical cross-entropy loss.
9. Train the model using the specified training configuration.
10. Evaluate the baseline model.
11. Perform automated hyperparameter optimization using RandomizedSearchCV with 5-fold cross-validation.
12. Retrain the model using the best hyperparameter configuration.
13. Evaluate the optimized model and compare it with the baseline model.

1.6 Dataset Exploration

1.6.1 Sample Images

The Fashion-MNIST dataset contains grayscale images with dimensions 28×28 . Ten representative samples from the dataset were visualized.

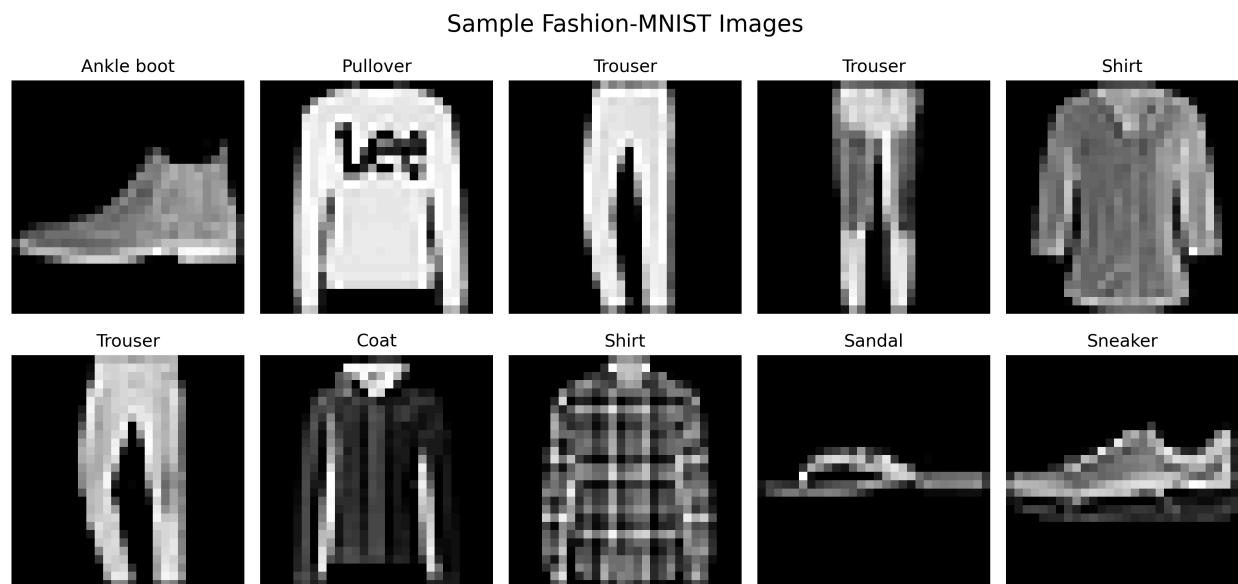


Figure 1: Sample images from the Fashion-MNIST dataset.

Inference: The sample images demonstrate the visual nature of the Fashion-MNIST dataset. Each image represents one of ten clothing categories and contains grayscale pixel information.

1.6.2 Class Distribution

The distribution of images across the ten Fashion-MNIST classes was visualized.

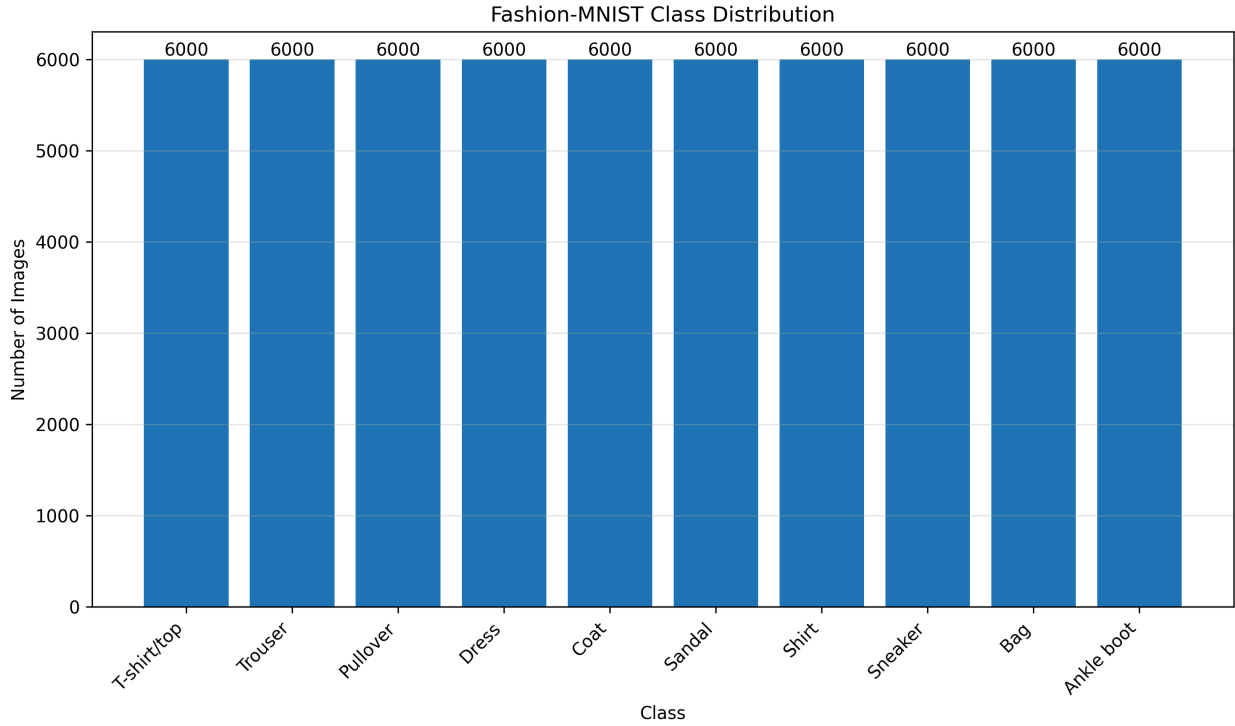


Figure 2: Class distribution of the Fashion-MNIST training dataset.

Inference: The class distribution is approximately balanced across the ten categories. Therefore, no major class imbalance is expected to dominate the training process.

1.7 Data Preprocessing

The original Fashion-MNIST images have dimensions 28×28 . Since dense layers require one-dimensional feature vectors, each image was flattened:

$$28 \times 28 = 784 \quad (5)$$

The pixel values were normalized from the original range $[0, 255]$ to $[0, 1]$. The class labels were converted into one-hot encoded vectors for multi-class classification.

1.8 Model Construction

The baseline MLP consists of an input layer containing 784 features, two hidden dense layers containing 128 and 64 neurons respectively, and an output layer containing 10 neurons.

Table 2: Baseline MLP Architecture

Layer	Neurons	Activation
Input Layer	784	–
Dense Layer 1	128	ReLU
Dense Layer 2	64	ReLU
Output Layer	10	Softmax

1.9 Model Training

The baseline model was compiled using the Adam optimizer and categorical cross-entropy loss. Accuracy was used as the primary training metric.

The training process was monitored using training and validation accuracy and loss.

1.10 Training Performance

1.10.1 Training and Validation Accuracy

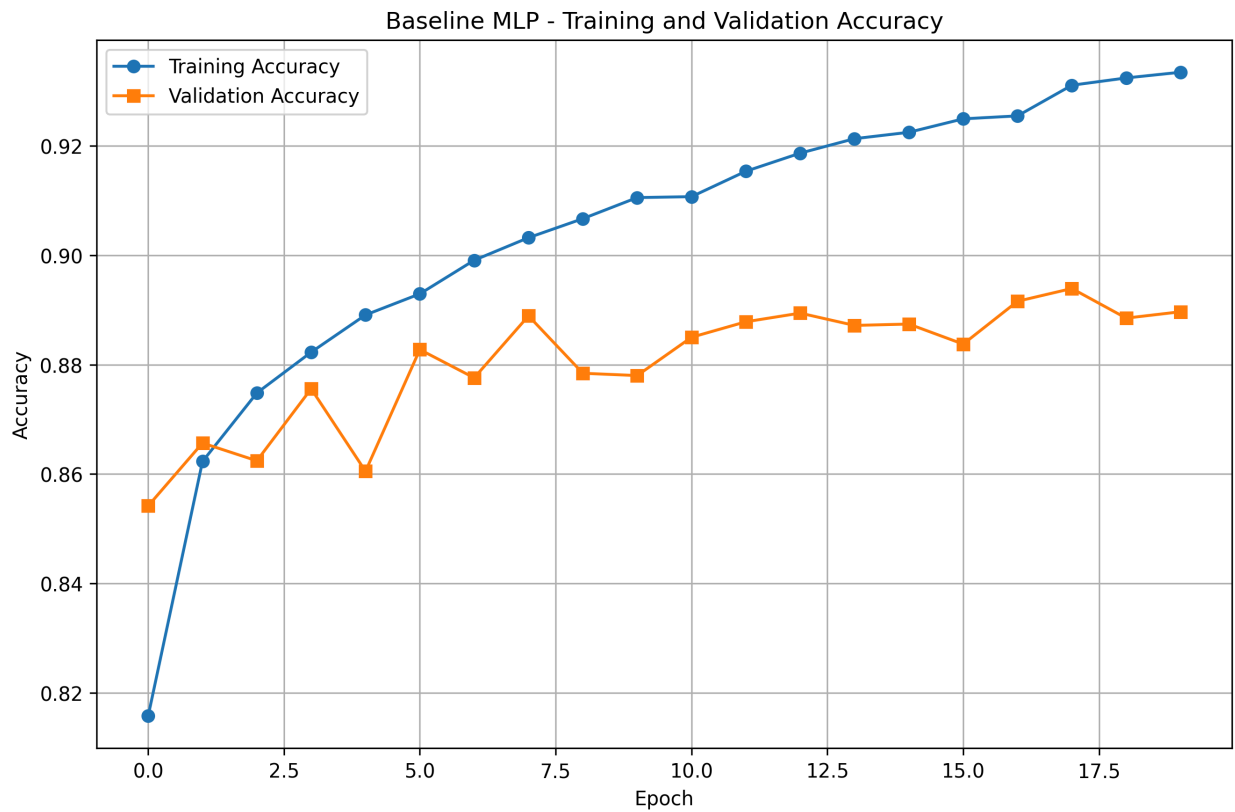


Figure 3: Training and validation accuracy of the baseline MLP.

Inference: The training accuracy generally increases as the number of epochs increases, indicating that the model learns useful features from the training data. The validation accuracy indicates how well the learned representation generalizes to unseen data.

1.10.2 Training and Validation Loss

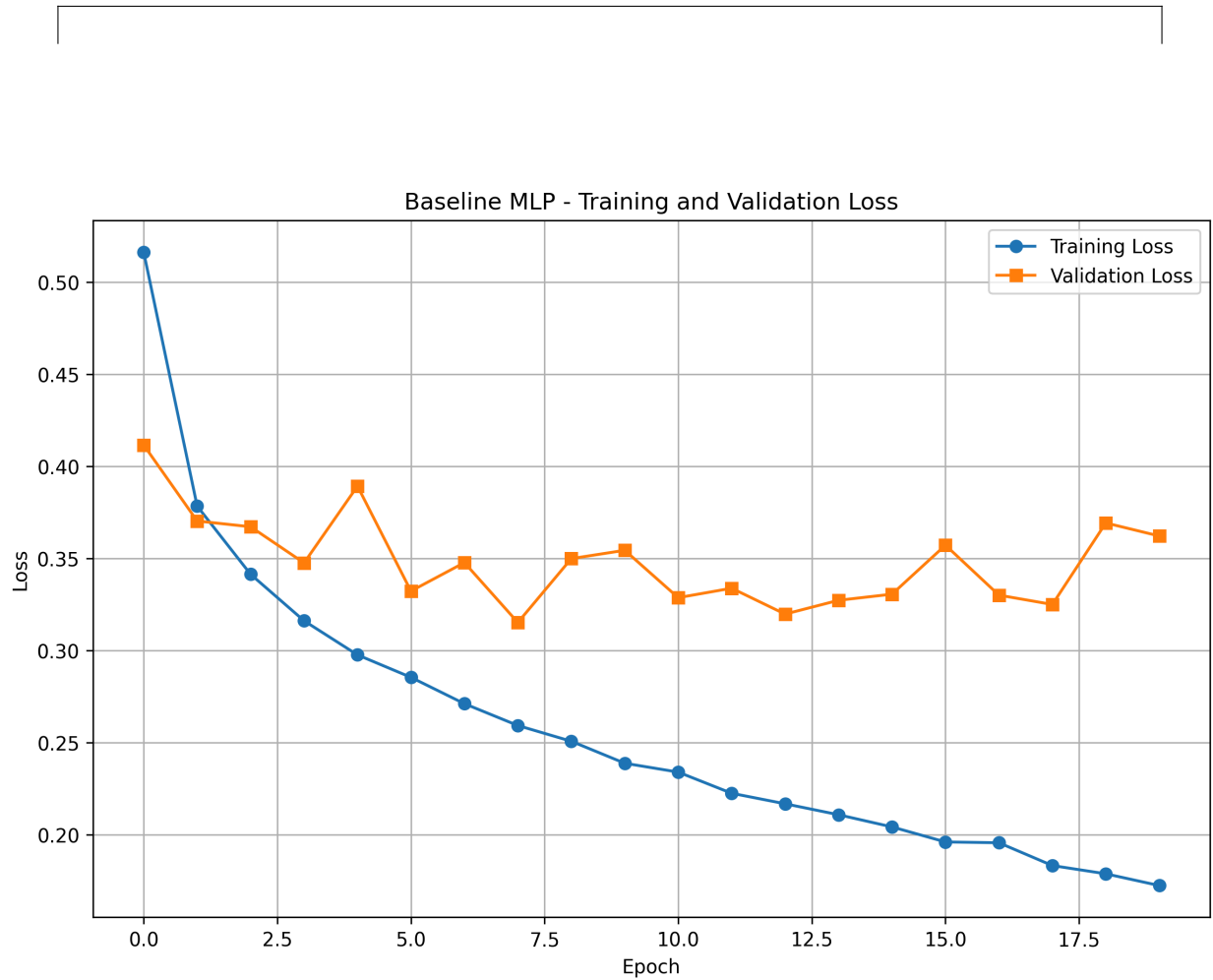


Figure 4: Training and validation loss of the baseline MLP.

Inference: The training loss decreases as the network learns the classification task. The validation loss provides an indication of the model's generalization performance and can also help identify potential overfitting.

1.11 Model Evaluation

The baseline MLP was evaluated using accuracy, precision, recall, F1-score, confusion matrix, and classification report.

Table 3: Baseline MLP Performance

Metric	Baseline MLP
Accuracy	[ENTER VALUE]
Precision	[ENTER VALUE]
Recall	[ENTER VALUE]
F1-Score	[ENTER VALUE]
Training Time	[ENTER VALUE] seconds

1.11.1 Confusion Matrix

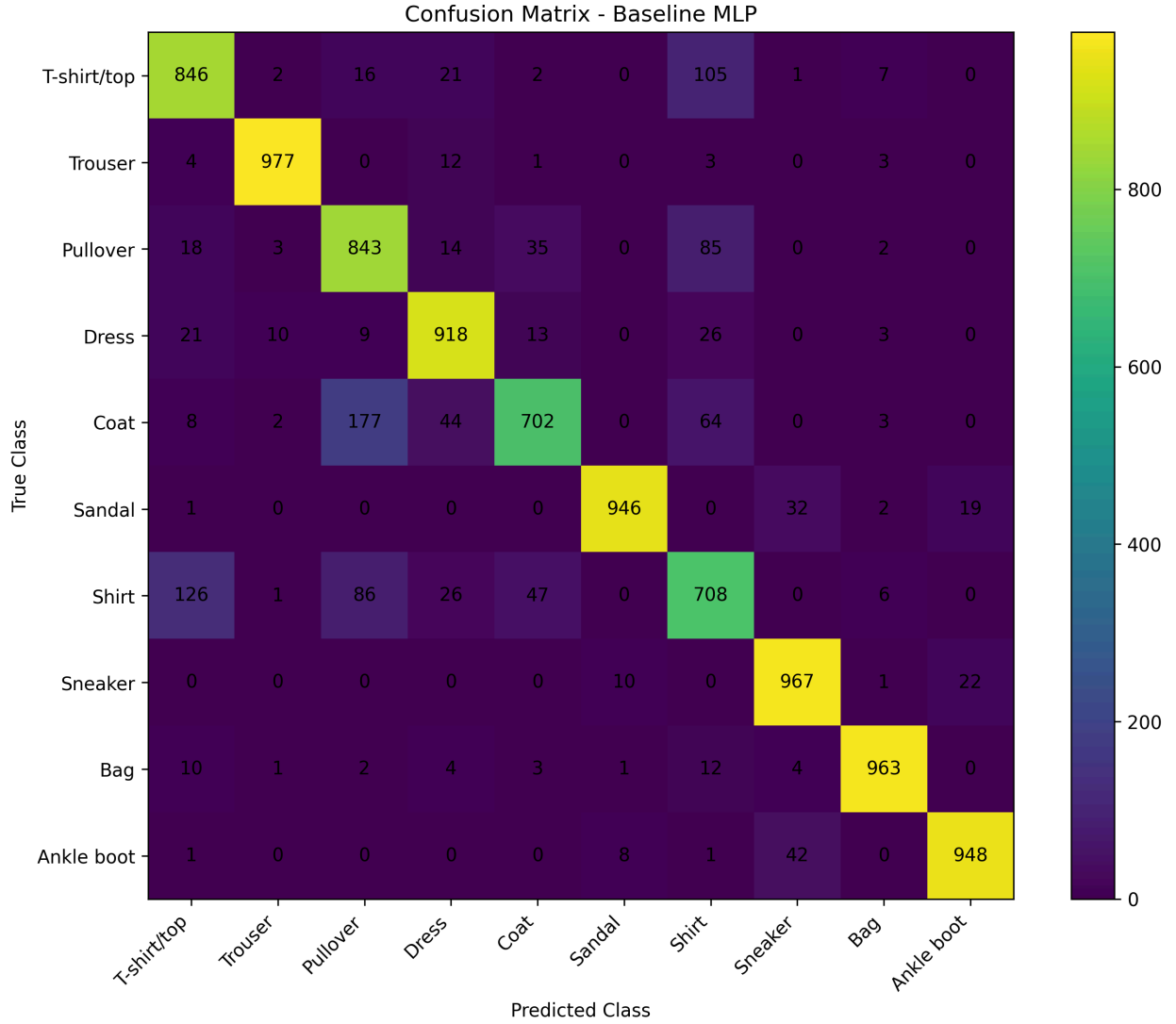


Figure 5: Confusion matrix of the baseline MLP.

Inference: The confusion matrix shows the classification performance for each Fashion-MNIST class. Most correctly classified samples appear along the main diagonal, while off-diagonal values represent misclassifications between visually similar clothing categories.

1.12 Hyperparameter Optimization

Automated hyperparameter optimization was performed using RandomizedSearchCV with the SciKeras wrapper and 5-fold cross-validation.

The search space included hidden layers, number of neurons, learning rate, batch size, epochs, optimizer, activation function, and dropout rate.

Table 4: Hyperparameter Search Space

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation	ReLU, Tanh, Sigmoid
Dropout	0.0, 0.2, 0.5

1.12.1 Hyperparameter Search Results

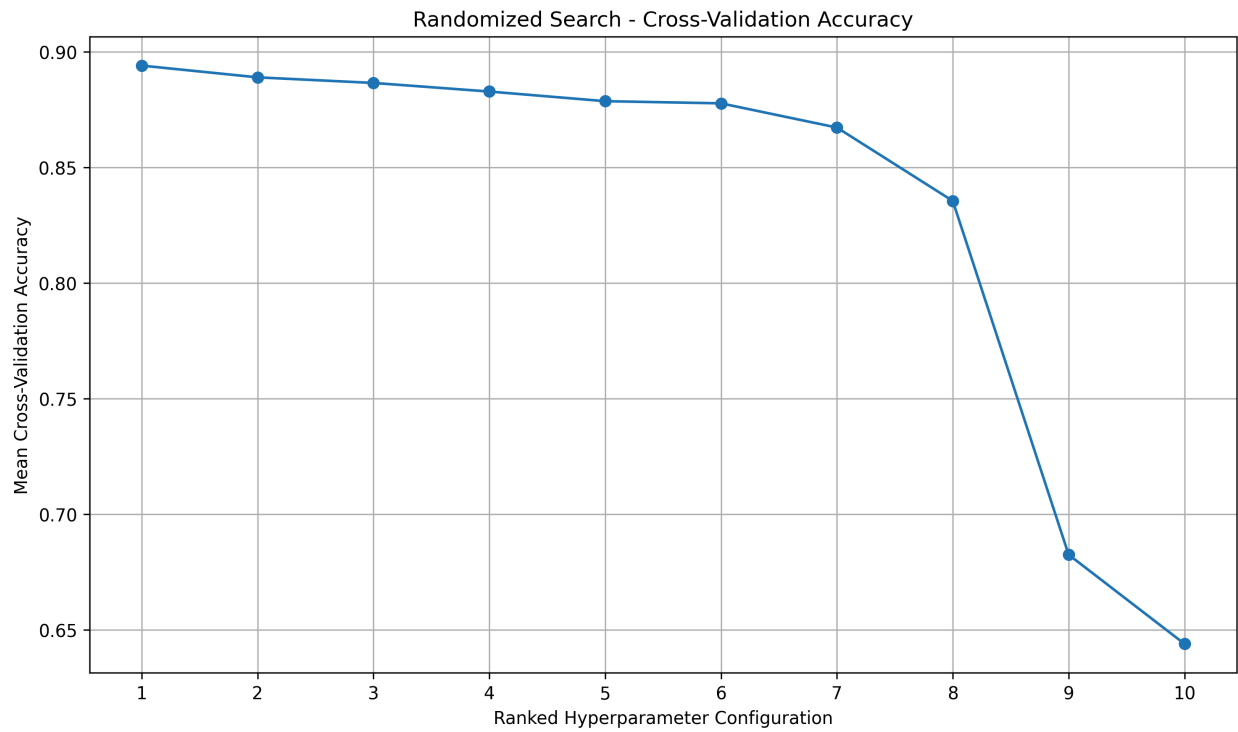


Figure 6: Cross-validation accuracy obtained during hyperparameter search.

Inference: The plot shows the variation in mean cross-validation accuracy across the tested hyperparameter configurations. The configuration with the highest cross-validation score was selected as the optimized model.

1.13 Best Hyperparameters

Table 5: Best Hyperparameter Configuration

Hyperparameter	Selected Value
Hidden Layers	[ENTER VALUE]
Hidden Neurons	[ENTER VALUE]
Learning Rate	[ENTER VALUE]
Batch Size	[ENTER VALUE]
Optimizer	[ENTER VALUE]
Activation Function	[ENTER VALUE]
Epochs	[ENTER VALUE]
Dropout	[ENTER VALUE]
Cross-validation Accuracy	[ENTER VALUE]
Testing Accuracy	[ENTER VALUE]

1.14 Optimized Model Training

The MLP was retrained using the best hyperparameter configuration obtained from RandomizedSearchCV.

1.14.1 Optimized Training and Validation Accuracy

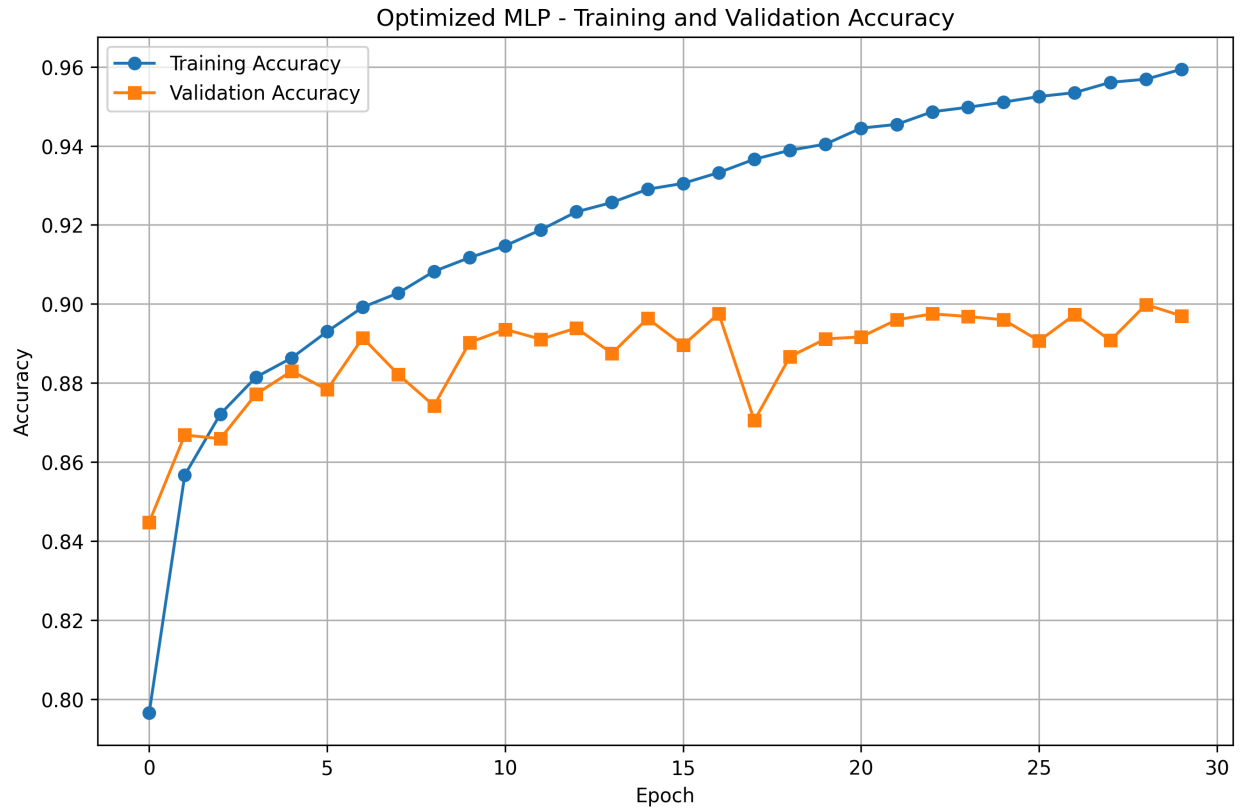


Figure 7: Training and validation accuracy of the optimized MLP.

Inference: The optimized model demonstrates the effect of the selected hyperparameters on the learning process. The relationship between training and validation accuracy provides an indication of the model's generalization capability.

1.14.2 Optimized Training and Validation Loss

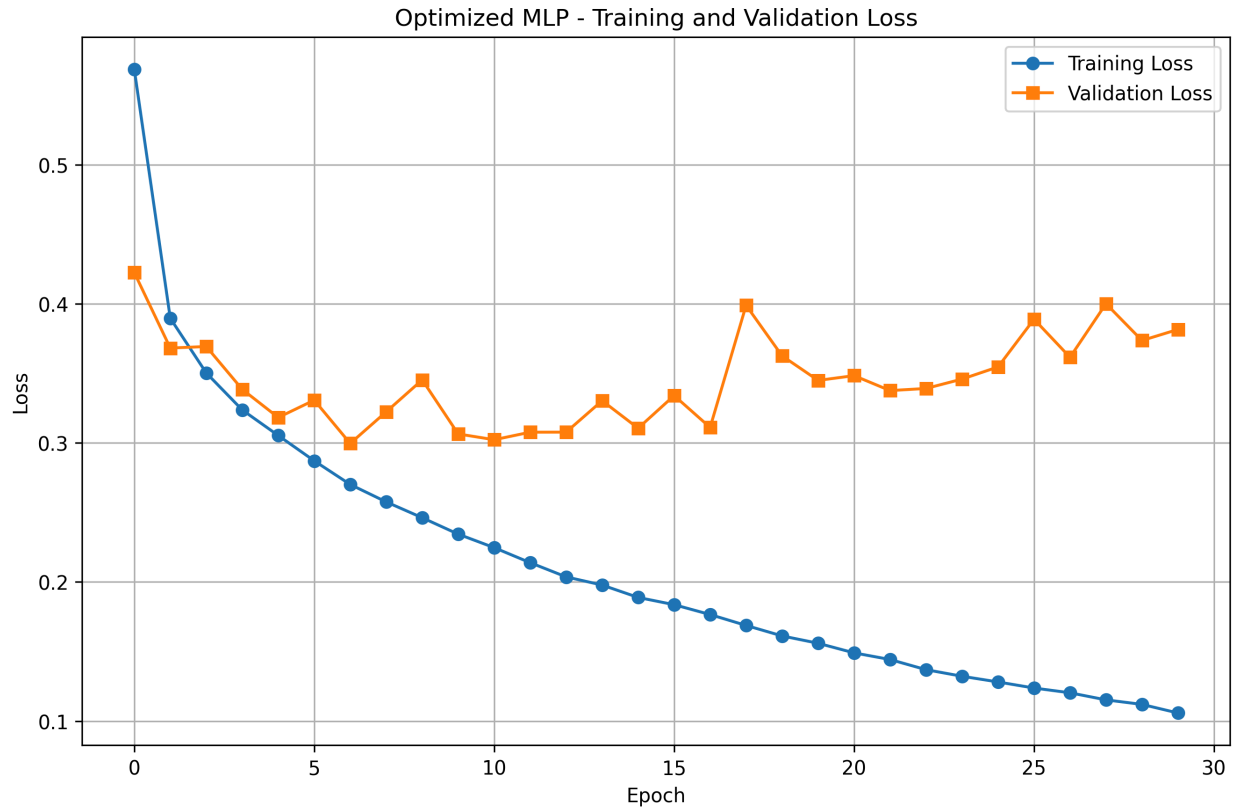


Figure 8: Training and validation loss of the optimized MLP.

Inference: The optimized model shows the variation of training and validation loss during learning. A decreasing training loss indicates effective optimization, while validation loss reflects performance on unseen validation data.

1.14.3 Optimized Confusion Matrix

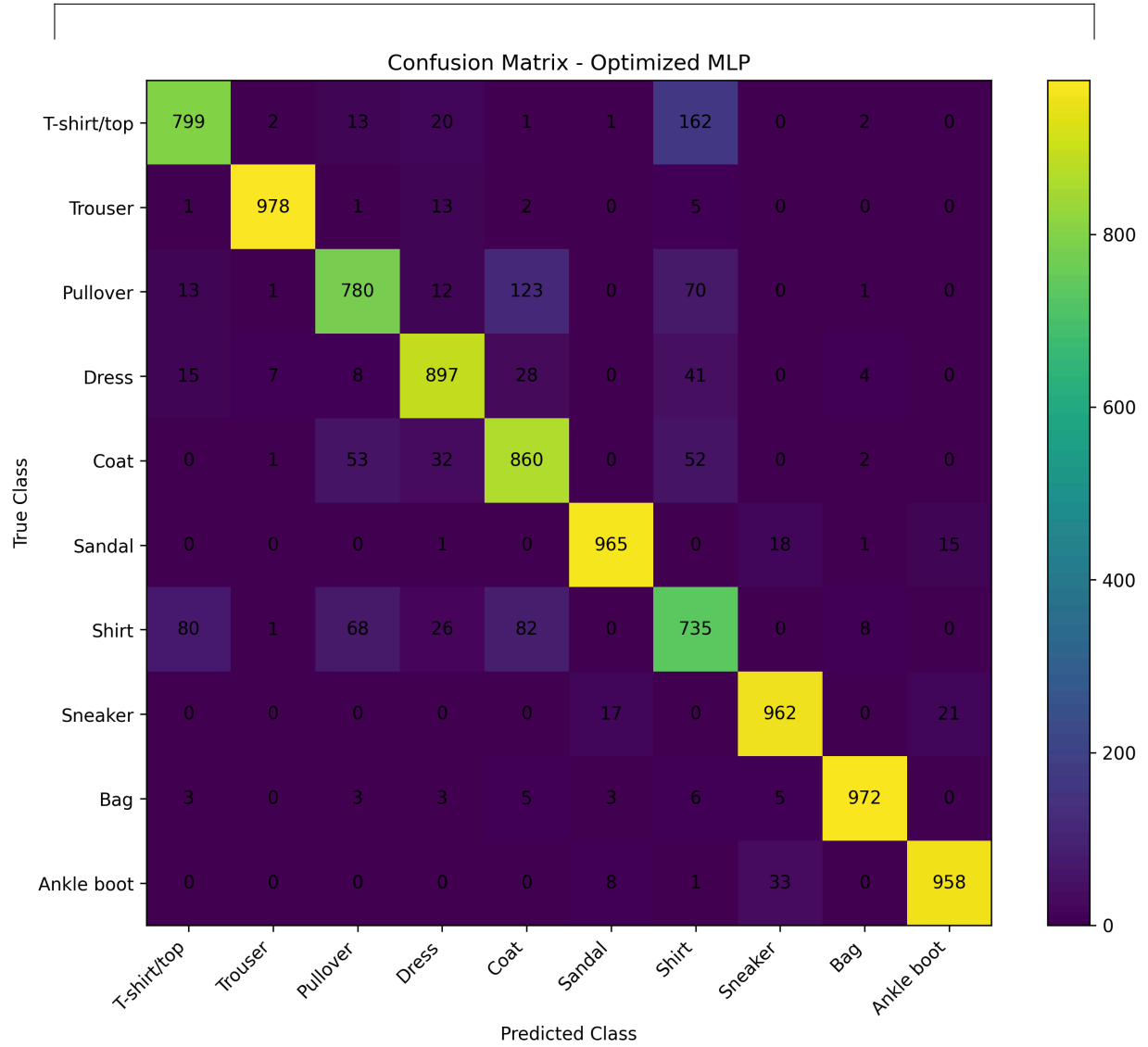


Figure 9: Confusion matrix of the optimized MLP.

Inference: The optimized confusion matrix provides a class-wise view of the final model performance. Comparison with the baseline confusion matrix helps identify whether classification errors were reduced after optimization.

1.15 Performance Comparison

Table 6: Baseline and Optimized MLP Performance Comparison

Metric	Baseline	Optimized
Accuracy	[ENTER VALUE]	[ENTER VALUE]
Precision	[ENTER VALUE]	[ENTER VALUE]
Recall	[ENTER VALUE]	[ENTER VALUE]
F1-Score	[ENTER VALUE]	[ENTER VALUE]
Training Time	[ENTER VALUE]	[ENTER VALUE]

1.15.1 Best Model Accuracy Comparison

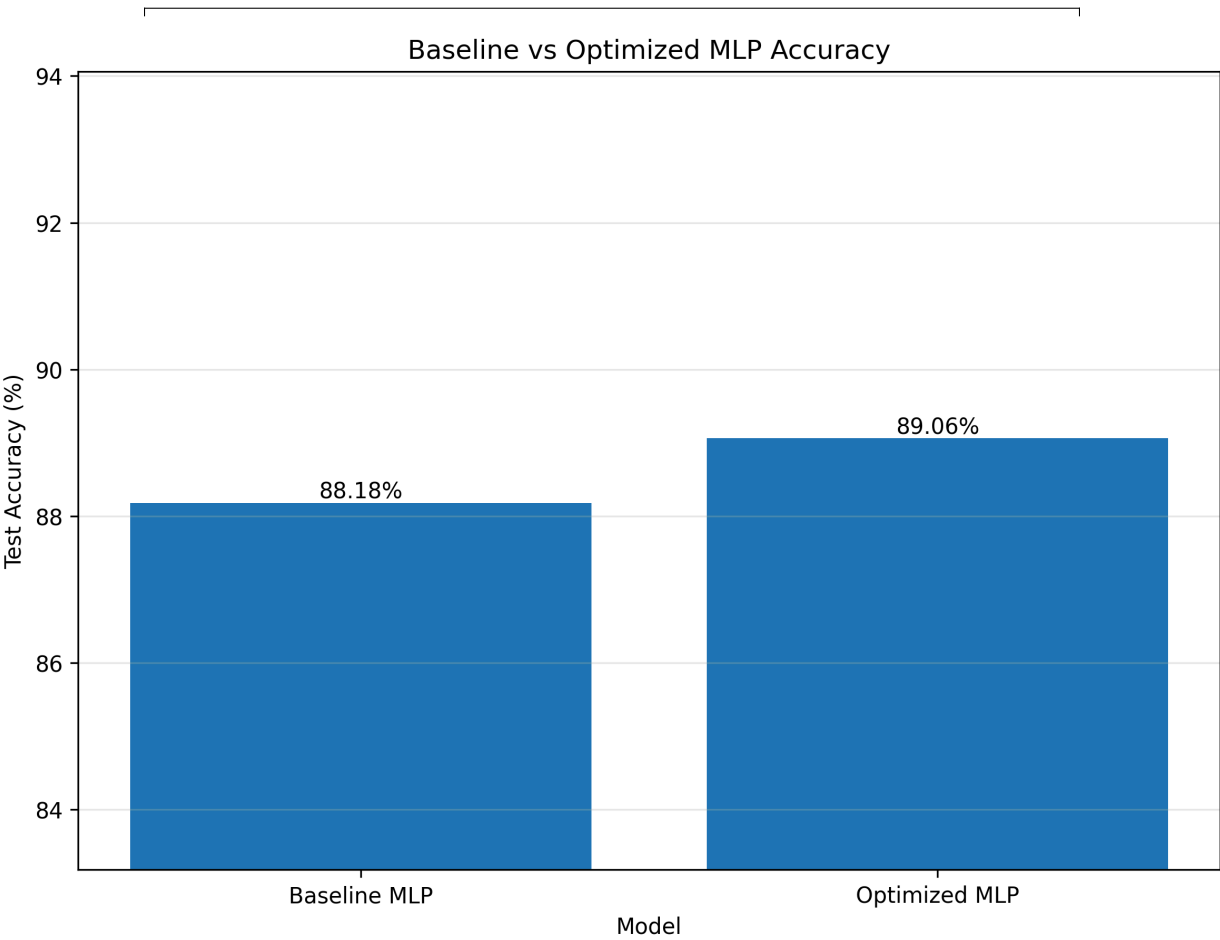


Figure 10: Comparison of baseline and optimized MLP test accuracy.

Inference: The accuracy comparison directly shows the effect of hyperparameter optimization. If the optimized model has higher test accuracy, optimization improved the model’s classification performance.

1.15.2 Overall Performance Comparison

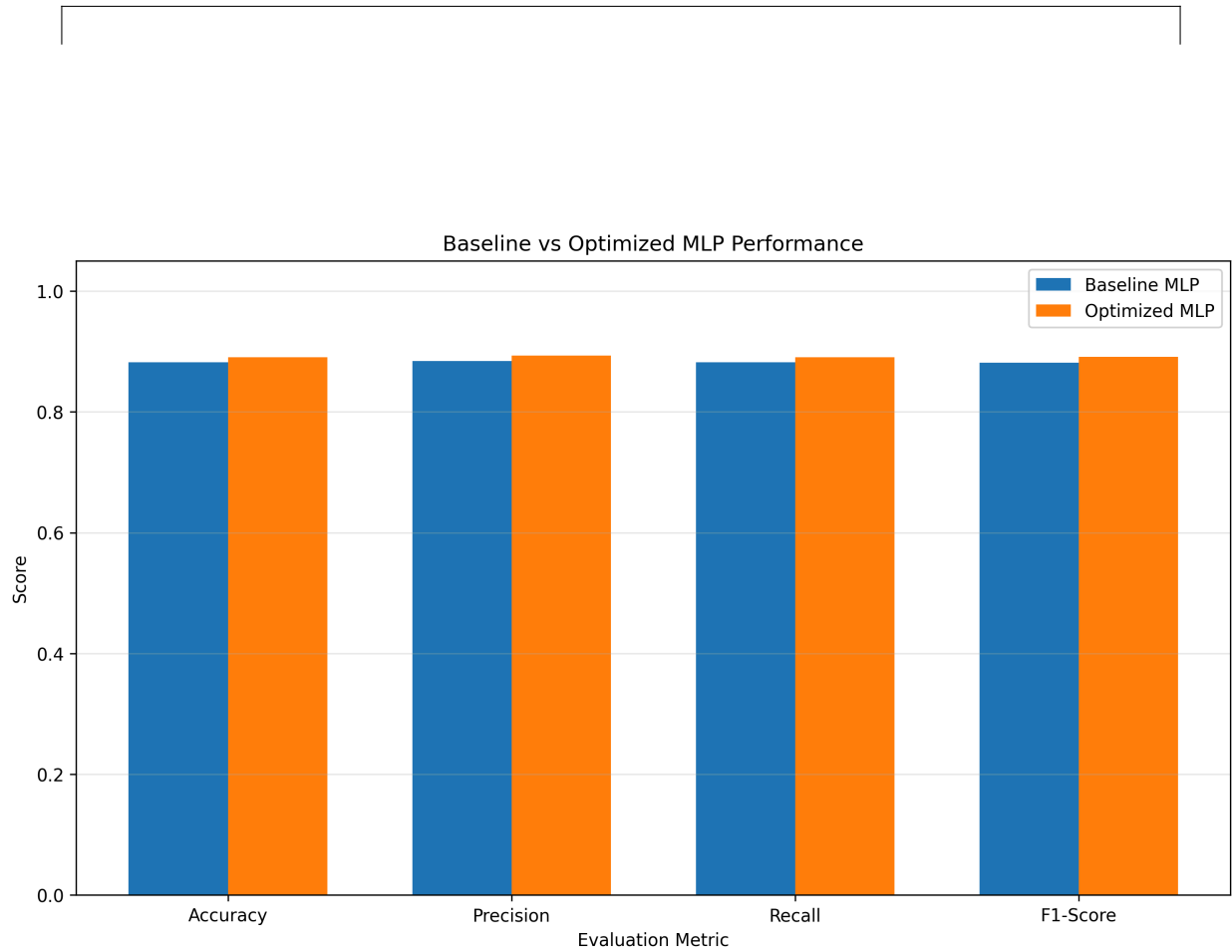


Figure 11: Comparison of baseline and optimized MLP performance metrics.

Inference: The comparison of accuracy, precision, recall, and F1-score provides a broader assessment of model performance. The optimized model can be considered preferable when it provides consistently better evaluation metrics.

1.16 Discussion

1. Which optimization method was used?

RandomizedSearchCV was used for automated hyperparameter optimization with 5-fold cross-validation.

2. Which hyperparameters were selected?

The best values were selected automatically based on the highest mean cross-validation accuracy. The selected values are presented in the Best Hyperparameter Configuration table.

3. Did optimization improve performance?

The optimized model was compared with the baseline model using accuracy, precision, recall, and F1-score. The improvement or decrease is determined from the experimental results.

4. Which hyperparameter had the greatest impact?

The hyperparameter associated with the largest change in cross-validation performance can be identified from the randomized search results.

5. Compare Grid Search and Randomized Search.

Grid Search evaluates every possible combination in the specified search space, making it computationally expensive for large search spaces. Randomized Search evaluates a selected number of randomly sampled combinations and can therefore explore a large search space more efficiently.

6. Recommend the best MLP configuration.

The configuration with the highest cross-validation performance and strong testing performance is recommended as the best MLP configuration for the Fashion-MNIST classification task.

1.17 Conclusion

The Multi-Layer Perceptron was successfully implemented for multi-class classification using the Fashion-MNIST dataset. The experiment demonstrated the complete workflow of image preprocessing, feature vector generation, MLP construction, model training, evaluation, and automated hyperparameter optimization.

RandomizedSearchCV with 5-fold cross-validation was used to identify an improved model configuration. The optimized MLP was subsequently evaluated on the testing dataset and compared with the baseline model using multiple performance metrics. The experiment demonstrates that appropriate selection of neural network architecture and hyperparameters can influence the classification performance of an MLP.